Cs621(Finals) - week 7 notes

Parallel and Distributed Computing (Virtual University of Pakistan)

Scan to open on Studocu

Studocu is not sponsored or endorsed by any college or university
Downloaded by Ali Zeeshan (az7168852@gmail.com)

Week 7:

Data Replication:

Data replication is the process by which data residing on a physical/virtual server or cloud instance is continuously replicated or copied to a secondary server (s) or cloud instance.

Utilization of data replication in an organization:

Organization uses the data replication for following purposes:

- **High Availability –** it ensures that the system accessible even during failures.
- **Backup –** protection against data loss.
- **Disaster Recovery –** enabling restoration of operations during catastrophic events.

Data reliability:

Data reliability means that data is complete and accurate. Replication increase the reliability of a system by:

- **Scaling in Numbers** – Adding more replicas to handle increased load.
- **Scaling Geographically** – Placing replicas closer to users to reduce latency (delay of data)

Advantage of data replication:

Replicas allow systems to continue functioning even in the event of local failures. They also protect against data corruption and improve performance by bringing data closer to where it is needed.

Challenges in Replication

While replication improves scalability and reliability, it introduces challenges such as:

- **Synchronization Overhead –** Keeping all replicas up-to-date requires significant communication and coordination.
- **Consistency Issues –** It's tricky to ensure all copies have the same data at the same time, especially when the replicas are far apart or there are many of them.
- **Relaxation of Consistency –** To improve performance, strict consistency constraints are often relaxed. (To make the system faster and easier to manage, we might allow the replicas to not always match perfectly for a short time. This means they won't always show the latest changes instantly, which can cause temporary inconsistencies.)

Data consistency:

Data consistency refers to the state of data in which all copies or instances are the same across all systems or servers.

Data consistency models:

Consistency ensures that all users and processes see the same data, regardless of which replica they access. A contract between processes and the data store that says that if processes agree to obey certain rules, the store promises to work correctly. Two broad categories of consistency models exist:

- Data-Centric Consistency Model
- Client-Centric Consistency Model

Data-Centric Consistency Model:

These focus on keeping all the replicas in sync so the data is the same everywhere.

Types of Data-Centric Consistency Models:

This document is available on studocu

Downloaded by Ali Zeeshan (az7168852@gmail.com)

Data-centric consistency models are frameworks that define how data should remain consistent across all replicas in a distributed system. The key data-centric consistency models include:

- Strict Consistency
- Sequential Consistency
- Casual Consistency
- Eventual Consistency
- Continuous Consistency
- Linearizability (Strong Consistency)

Strict Consistency:

Every time you read a data, you always get the latest update instantly, no matter which copy you use.

Example:

If someone changes its profile picture, you see the change immediately on all devices.

Sequential Consistency:

Everyone sees all operations (reads and writes) in the same order, but that order doesn't have to match real-time.

Example:

If two users edit a shared document, everyone will see the changes in the same sequence, even if it's not the exact order in which they happened.

Casual Consistency:

If one operation depends on another, they must happen in the right order on all replicas. Operations that are unrelated can happen in any order.

Example:

If you reply to a message, the message and the reply must appear in the correct order for everyone. But if two people send unrelated messages, the order doesn't matter.

Eventual Consistency:

If no new changes are made, all copies will eventually become the same, but they might differ temporarily.

Example:

If no new changes are made, all copies will eventually become the same, but they might differ temporarily.

Continuous Consistency:

Instead of being either fully consistent or inconsistent, it measures *how much* the replicas differ. This can include:

- How different the values are.
- How far behind the updates are.
- Whether the updates are in the right order.

Example:

A stock price app might allow small delays or differences between replicas as long as they stay within a certain limit.

Linearizability (Strong consistency):

Downloaded by Ali Zeeshan (az7168852@gmail.com)

It’s like strict consistency but focuses on making operations look like they happen instantly at a single point in time.

Example:

If you buy a ticket online, the system ensures no one else can buy the same ticket, even for a brief moment.

Client-Centric Consistency Model:

These models focus on making sure each user has a smooth experience, even if the replicas are not perfectly in sync. It’s more about what the user sees than the actual state of all the replicas.

Types of client-centric consistency model:

These models are particularly useful in systems with eventual consistency, where global consistency is not guaranteed immediately. The key client-centric consistency models include:

- Monotonic Reads
- Monotonic Writes
- Read your writes
- Writes Follow by reads

Monotonic Reads:

If you read a value from a system, the next time you read it, you will either get the same value or a more recent one. You won’t get an older value after seeing a newer one.

Example:

A user reads a social media post (version 2). Later, they will see version 2 or a newer version (e.g., version 3), but never an older version (e.g., version 1).

Monotonic Writes:

If a client performs a write operation, any later writes by the same client will happen in the same order as they were issued.

Example:

A user edits a document twice. If the system processes the first edit (write), it ensures the second edit happens afterward, maintaining the order of changes.

Read your write:

After a client writes data, any subsequent reads by the same client will reflect the result of their write.

Example:

A user posts a comment on a blog. If they refresh the page, they should see their comment, even if other replicas haven't updated yet.

Writes Follow Reads:

If a client reads a value and then writes a related update, the write will be based on the value they read.

Example:

A user sees a document version (v1) and makes an update. The update will apply to version v1, ensuring consistency with what they last saw.

This document is available on studocu

Downloaded by Ali Zeeshan (az7168852@gmail.com)